NGHIÊN CỨU MÔ HÌNH HÓA SAT ENCODING CHO RÀNG BUỘC CARDINALITY (AMO/ALO) VÀ ĐÁNH GIÁ THỰC NGHIỆM SO SÁNH GIẢI SUDOKU $N \times N$ VỚI CÁC PHƯƠNG PHÁP CHÍNH XÁC (CP, ILP/MIP)

Nguyễn Minh Sơn[a,*]

[a]*Khoa công nghệ thông tin, Trường đại học Công nghệ*

ARTICLE INFO

Keywords:
Sudoku Solver
SAT Encoding
AMO Constraints
Constraint Programming
Integer Linear Programming
PySAT
Glucose3
OR-Tools
Gurobi
CPLEX

Tóm tắt nội dung

Báo cáo trình bày việc mô hình hóa và giải Sudoku bằng SAT, tập trung vào các ràng buộc At-Least-One (ALO), At-Most-One (AMO) và Exactly-One (EO). Hệ thống tích hợp năm kỹ thuật AMO: Pairwise, Binary, Commander, 2-Product và Sequential Counter; sử dụng Glucose3 thông qua PySAT và so sánh với OR-Tools CP-SAT, IBM CP Optimizer, Gurobi, CPLEX MIP và PuLP/CBC. Bộ dữ liệu đóng gói gồm 299 bản ghi ở ba kích thước 4×4, 9×9, 16×16, tương ứng 298 đề phân biệt, lấy từ Peter Norvig, Sudoku Exchange và dữ liệu tự sinh. Báo cáo phân tích quy mô CNF, chi phí mã hóa và thời gian thực thi hàm giải; đồng thời mô tả kiến trúc chương trình, giao diện dòng lệnh và kiểm tra nghiệm. Các số liệu đã ghi nhận cho thấy lựa chọn encoding và phạm vi đo ảnh hưởng đến thứ hạng phương pháp. Kết luận được giới hạn trong tập dữ liệu và cấu hình khảo sát; các hạn chế về dữ liệu, đo lường và khả năng tái lập được trình bày riêng.

1. Giới thiệu (Introduction)

Sudoku là bài toán điền giá trị vào bảng sao cho mỗi hàng, cột và khối con chứa các giá trị không lặp. Với bảng chuẩn 9×9, miền giá trị là 1 đến 9 và mỗi khối có kích thước 3×3. Bài toán có thể mở rộng sang bảng $N \times N$, với $N = k^2$ đối với khối vuông $k \times k$. Đây là một ví dụ phù hợp để khảo sát mô hình hóa bài toán thỏa mãn ràng buộc; Norvig trình bày cách giải bằng lan truyền ràng buộc và tìm kiếm Norvig (2006).

SAT (Boolean Satisfiability Problem) xác định sự tồn tại của phép gán làm đúng công thức Boole. Các bộ giải hiện đại như Glucose sử dụng CDCL (Conflict-Driven Clause Learning) Audemard and Simon (2009). Khi áp dụng SAT cho Sudoku, bước quan trọng là chuyển các ràng buộc sang dạng chuẩn hội CNF (Conjunctive Normal Form). Báo cáo tập trung vào ALO (ít nhất một biến đúng), AMO (nhiều nhất một biến đúng) và EO (đúng một biến đúng).

Đồng thời, bên cạnh tiếp cận bằng SAT, bài toán Sudoku còn có thể được mô hình hóa và giải quyết bằng các phương pháp toán học chính xác khác như:

- **Constraint Programming (CP)**: Khai thác các ràng buộc toàn cục như `AllDifferent` thông qua lan truyền miền giá trị cục bộ (domain filtering), tiêu biểu là Google OR-Tools CP-SAT Perron and Furnon (2024) và IBM ILOG CPLEX CP Optimizer IBM Corporation (2024).
- **Integer Linear Programming (ILP / MIP)**: Sử dụng quy hoạch tuyến tính nguyên với các biến nhị phân $\{0, 1\}$, giải quyết bằng phương pháp nhánh cận (Branch-and-Bound) kết hợp vết cắt phẳng (Cutting Planes), tiêu biểu là Gurobi Optimizer Gurobi Optimization, LLC (2024), IBM CPLEX MIP, và CBC Solver.

Mục tiêu của bài tập lớn là:

1. Xây dựng hệ thống giải Sudoku tích hợp năm kỹ thuật AMO, phân biệt phần tự cài đặt và phần sử dụng thư viện.
2. So sánh Glucose3 với năm cấu hình bộ giải CP và ILP/MIP trên cùng đầu vào.
3. Phân tích quy mô mô hình, thời gian thực thi và tính đúng đắn của nghiệm; nêu rõ phạm vi và giới hạn thực nghiệm.

*Tác giả liên hệ.
24021614@vnu.edu.vn (N.M. Sơn)
ORCID(s):

2. Mô hình hóa bài toán Sudoku bằng SAT (SAT Formulation)

Xét bảng Sudoku kích thước $N \times N$ với các khối con kích thước $k_r \times k_c$ ($N = k_r \times k_c$). Mỗi ô trong bảng được định vị bởi tọa độ (r, c) với $0 \le r, c < N$. Các giá trị điền vào nhận giá trị $d \in \{1, 2, \dots, N\}$.

2.1. Biểu diễn biến mệnh đề

Ta định nghĩa biến Boole có ba chỉ số $x_{r,c,d}$:

$$x_{r,c,d} = \begin{cases} 1 & \text{nếu ô } (r,c) \text{ nhận chữ số } d \\ 0 & \text{ngược lại} \end{cases} \tag{1}$$

Số lượng biến Boole ban đầu cho bảng kích thước $N \times N$ là $N \times N \times N = N^3$ biến. Trong cài đặt mã nguồn, chỉ số biến (1-based index) được ánh xạ theo công thức song ánh:

$$\text{ID}(r, c, d) = N^2 \cdot r + N \cdot c + d \quad (1 \le \text{ID} \le N^3) \tag{2}$$

2.2. Hệ thống ràng buộc của Sudoku

Một cấu hình Sudoku hợp lệ phải thỏa mãn đồng thời 5 nhóm ràng buộc logic sau:

1. **Ràng buộc ô (Cell Constraints)**: Mỗi ô (r, c) phải nhận *đúng một* chữ số từ 1 đến N:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 1 \le d \le N\}\Big), \quad \forall 0 \le r, c < N \tag{3}$$

2. **Ràng buộc hàng (Row Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trên mỗi hàng r:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 0 \le c < N\}\Big), \quad \forall 0 \le r < N,\ 1 \le d \le N \tag{4}$$

3. **Ràng buộc cột (Column Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trên mỗi cột c:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 0 \le r < N\}\Big), \quad \forall 0 \le c < N,\ 1 \le d \le N \tag{5}$$

4. **Ràng buộc khối con (Block/Box Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trong mỗi khối (br, bc):

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid r \in \text{block}_r(br), c \in \text{block}_c(bc)\}\Big) \tag{6}$$

5. **Ràng buộc gợi ý ban đầu (Clue / Given Constraints)**: Nếu ô (r, c) đã có giá trị cho trước $v \in \{1, \dots, N\}$, mệnh đề đơn nguyên (unit clause) được bổ sung:

$$(x_{r,c,v}) \quad \forall (r, c, v) \in \text{Clues} \tag{7}$$

Một ràng buộc Exactly-One (EO) trên tập biến $X = \{x_1, \dots, x_n\}$ được phân rã thành:

$$\text{EO}(X) \iff \text{At-Least-One}(X) \wedge \text{At-Most-One}(X) \tag{8}$$

Ràng buộc **At-Least-One (ALO)** được biểu diễn bằng một mệnh đề tuyển:

$$\text{ALO}(X) = (x_1 \vee x_2 \vee \dots \vee x_n) \tag{9}$$

Trong khi đó, việc mã hóa ràng buộc **At-Most-One (AMO)** có ảnh hưởng quyết định đến quy mô của file CNF và thời gian giải của SAT Solver.

3. Các kỹ thuật mã hóa At-Most-One (AMO Encodings)

Cho $X = \{x_1, \dots, x_n\}$, AMO yêu cầu $\sum_{i=1}^{n} x_i \le 1$. Năm kỹ thuật dưới đây được tích hợp trong hệ thống. Các công thức đếm áp dụng cho $n \ge 2$ theo cấu trúc được mô tả; trường hợp nhỏ có thể được thư viện rút gọn.

3.1. Pairwise Encoding (Naive Approach)

Phương pháp tổ hợp cặp đôi cổ điển phát biểu rằng hai biến bất kỳ trong X không thể đồng thời nhận giá trị True:

$$\mathrm{AMO}_{\mathrm{pair}}(X) = \bigwedge_{1 \le i < j \le n} (\neg x_i \vee \neg x_j) \tag{10}$$

- **Số biến phụ**: 0.
- **Số mệnh đề**: $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Đặc điểm**: Dễ cài đặt, không làm tăng không gian biến tìm kiếm nhưng số mệnh đề bùng nổ theo hàm bậc hai khi n tăng lớn.

3.2. Binary Encoding (Bitwise Approach)

Phương pháp sử dụng $m = \lceil \log_2 n \rceil$ biến phụ $b_1, \dots, b_m$ để đánh số nhị phân các biến đầu vào; xem thảo luận về mã hóa mệnh đề trong Frisch et al. (2008). Biến x_i tương ứng với mã của $i-1$. Khi $x_i = 1$, các bit phụ phải nhận đúng mã này:

$$\mathrm{AMO}_{\mathrm{bin}}(X) = \bigwedge_{i=1}^{n} \bigwedge_{j=1}^{m} \Big(\neg x_i \vee \ell(i,j) \Big) \tag{11}$$

trong đó $\ell(i,j) = b_j$ nếu bit thứ j của $(i-1)$ là 1, và $\ell(i,j) = \neg b_j$ nếu bit thứ j là 0.

- **Số biến phụ**: $\lceil \log_2 n \rceil = O(\log n)$.
- **Số mệnh đề**: $n\lceil \log_2 n \rceil = O(n \log n)$.
- **Đặc điểm**: Tiết kiệm biến phụ và mệnh đề đáng kể khi n rất lớn.

3.3. Commander Variable Encoding

Kỹ thuật do Klieber and Kwon (2007) đề xuất. Ý tưởng chính là phân hoạch n biến thành các nhóm con kích thước k (thường chọn $k = 3$). Mỗi nhóm được chỉ định một biến chỉ huy (*commander variable*) c_g:

1. Bên trong mỗi nhóm con, áp dụng Pairwise AMO: $\bigwedge_{u<v}(\neg u \vee \neg v)$.
2. Ràng buộc biến phần tử với biến chỉ huy: $x \to c_g$ $(\neg x \vee c_g)$.
3. Ràng buộc biến chỉ huy với nhóm: $c_g \to \bigvee_{x \in g} x$ $(\neg c_g \vee \bigvee x)$.
4. Áp dụng đệ quy ràng buộc AMO trên tập biến chỉ huy $\{c_1, c_2, \dots\}$.

- **Số biến phụ**: $O(n/k) = O(n)$.
- **Số mệnh đề**: $O(n)$ (tuyến tính cả về biến và mệnh đề).

3.4. 2-Product Encoding (Jingchao Chen, 2010)

Phương pháp 2-Product được Chen (2010) đề xuất, bố trí n biến vào một lưới 2 chiều kích thước $p \times q$ sao cho $p \times q \ge n$, với $p = \lceil \sqrt{n} \rceil$ và $q = \lceil n/p \rceil$. Ta giới thiệu p biến hàng $r_1, \dots, r_p$ và q biến cột $c_1, \dots, c_q$. Mỗi biến $x_{i,j}$ tương ứng với ô ở hàng i, cột j:

$$x_{i,j} \to r_i \iff (\neg x_{i,j} \vee r_i) \tag{12}$$

$$x_{i,j} \to c_j \iff (\neg x_{i,j} \vee c_j) \tag{13}$$

$$\mathrm{AMO}(r_1, \dots, r_p) \wedge \mathrm{AMO}(c_1, \dots, c_q) \tag{14}$$

- **Số biến phụ**: $p + q = O(\sqrt{n})$; bằng $2\sqrt{n}$ khi n là số chính phương, không bằng $2\lceil \sqrt{n} \rceil$ trong mọi trường hợp.
- **Số mệnh đề**: $2n + \binom{p}{2} + \binom{q}{2} = O(n)$ khi dùng Pairwise trên các biến hàng và cột như cài đặt hiện tại.
- **Đặc điểm**: Cấu trúc lưới giảm số mệnh đề khi n đủ lớn nhưng phát sinh chi phí bổ sung ở nhóm nhỏ.

3.5. Sequential Counter Encoding

Được Sinz (2005) đề xuất, sử dụng một chuỗi biến đếm $s_1, s_2, \ldots, s_{n-1}$ đại diện cho trạng thái tích lũy của chuỗi tiền tố:

$$(\neg x_1 \vee s_1) \tag{15}$$

$$(\neg x_i \vee s_i), \quad (\neg s_{i-1} \vee s_i), \quad (\neg x_i \vee \neg s_{i-1}) \quad \forall 2 \le i \le n-1 \tag{16}$$

$$(\neg x_n \vee \neg s_{n-1}) \tag{17}$$

- **Số biến phụ**: $n - 1$.
- **Số mệnh đề**: $3n - 4 = O(n)$.
- **Lan truyền đơn vị**: Khi một biến đầu vào đúng, chuỗi biến phụ cho phép suy ra các biến đầu vào khác sai mà không cần phân nhánh thêm để thực hiện suy diễn này.

Trong bảng dưới, lan truyền được xét trên biến đầu vào của AMO: một biến đúng suy ra các biến còn lại sai, hai biến đúng dẫn đến xung đột. Binary cũng có tính chất này vì hai mã khác nhau có ít nhất một bit khác nhau. Tính chất này không có nghĩa mọi encoding có chi phí lan truyền hoặc tìm kiếm như nhau.

Bảng 1
So sánh đặc tính lý thuyết của 5 phương pháp mã hóa At-Most-One (AMO).

Phương pháp AMO	Tác giả / Nguồn	Số biến phụ	Số mệnh đề	Đặc tính Unit Propagation
Pairwise (Naive)	Cổ điển	0	$\frac{n(n-1)}{2} = O(n^2)$	Suy diễn trực tiếp
Binary (Bitwise)	Frisch et al. (2008)	$\lceil \log_2 n \rceil$	$n\lceil \log_2 n \rceil = O(n \log n)$	Suy diễn qua các bit phụ
Commander	Klieber and Kwon (2007)	$O(n)$, k cố định	$O(n)$	Suy diễn trong và giữa nhóm
2-Product	Chen (2010)	$p + q = O(\sqrt{n})$	$2n + \binom{p}{2} + \binom{q}{2} = O(n)$	Qua biến hàng và cột
Sequential Counter	Sinz (2005)	$n - 1$	$3n - 4 = O(n)$	Qua chuỗi biến đếm

4. Các phương pháp giải đối chuẩn khác (Alternative Approaches)

Để đối chiếu với SAT, hệ thống mô hình hóa Sudoku trên năm cấu hình bộ giải CP và ILP/MIP:

4.1. Google OR-Tools CP-SAT Solver

CP-SAT là bộ giải Constraint Programming kết hợp SAT hiện đại phát triển bởi Google Perron and Furnon (2024). Trong CP-SAT, Sudoku được mô hình hóa tự nhiên bằng mảng 2 chiều các biến nguyên $y_{r,c} \in \{1, 2, \ldots, N\}$. Các ràng buộc hàng, cột và khối con được biểu diễn thông qua ràng buộc toàn cục:

$$\texttt{AddAllDifferent}\Big(\{y_{r,c} \mid c = 0 \ldots N-1\}\Big) \tag{18}$$

Mã nguồn khai báo `AddAllDifferent` cho hàng, cột và khối, cùng đẳng thức cho gợi ý. Việc tiền xử lý và lựa chọn cơ chế lan truyền do OR-Tools quyết định; báo cáo không giả định một thuật toán lọc miền cụ thể luôn được kích hoạt.

4.2. IBM ILOG CPLEX CP Optimizer

Bộ giải CP của IBM sử dụng biến `integer_var(1, N)`, ràng buộc `all_diff()` cho hàng, cột, khối và đẳng thức cho gợi ý. Hệ thống gọi bộ máy của CP Optimizer, không tự cài đặt hoặc cấu hình chiến lược tìm kiếm cụ thể.

4.3. Integer Linear Programming (ILP/MIP): Gurobi, CPLEX MIP & PuLP/CBC

Trong mô hình Quy hoạch tuyến tính nguyên:

- Biến quyết định nhị phân: $z_{r,c,d} \in \{0, 1\}$.
- Hàm mục tiêu: Tìm nghiệm thỏa mãn ràng buộc (Hàm mục tiêu bằng 0 hoặc hằng số): min 0.

- Hệ thống ràng buộc đẳng thức tuyến tính:

$$\sum_{d=1}^{N} z_{r,c,d} = 1 \quad \forall r, c \tag{19}$$

$$\sum_{c=0}^{N-1} z_{r,c,d} = 1 \quad \forall r, d \tag{20}$$

$$\sum_{r=0}^{N-1} z_{r,c,d} = 1 \quad \forall c, d \tag{21}$$

$$\sum_{r \in \text{block}} \sum_{c \in \text{block}} z_{r,c,d} = 1 \quad \forall \text{block}, d \tag{22}$$

Mô hình được giải bằng Gurobi, CPLEX MIP hoặc CBC thông qua PuLP. Gợi ý được cố định bằng $z_{r,c,v} = 1$. Đây là bài toán tìm nghiệm khả thi, không phải tối ưu chất lượng lời giải Sudoku.

5. Thiết kế và cài đặt hệ thống

5.1. Kiến trúc và luồng xử lý

Chương trình được tổ chức thành các mô-đun độc lập: `sat_encodings/` sinh ALO, AMO và EO; `solvers/` xây dựng mô hình và gọi bộ giải; `benchmarks/` nạp dữ liệu, đo thời gian và vẽ biểu đồ; `tests/` kiểm thử; `main.py` điều phối giao diện dòng lệnh; `utils.py` cung cấp bảng mẫu, suy ra kích thước khối và in bảng.

Luồng benchmark gồm: đọc snapshot và lọc đề; xây dựng CNF hoặc mô hình CP/MIP; gọi solver; giải mã kết quả; kiểm tra hàng, cột, khối và việc giữ nguyên gợi ý; tổng hợp thống kê rồi xuất kết quả. SAT sử dụng N^3 biến cơ sở và cấp các ID phụ phía sau để tránh trùng biến. Pairwise, Commander và Product tự sinh mệnh đề bằng Python; Binary và Sequential Counter gọi `CardEnc.atmost` của PySAT Ignatiev et al. (2018) với kiểu mã hóa tương ứng.

5.2. Chức năng và cách sử dụng

CLI hỗ trợ kích thước 4, 9 và 16; giải một bảng mẫu bằng SAT; benchmark nhanh trên bảng mẫu hoặc benchmark tập dữ liệu; lọc theo kích thước, nguồn và nhãn độ khó; giới hạn số bản ghi; xuất CSV/JSON, biểu đồ và tên tệp có dấu thời gian. API Python cho phép truyền ma trận đề trực tiếp, còn chế độ giải đơn lẻ của CLI hiện chỉ chọn bảng mẫu và encoding, chưa có chức năng nhập đề tùy ý qua tệp.

```
pip install -r requirements.txt
python main.py --size 9 --solve product
python main.py --benchmark-dataset-encodings --size 9 --plot --csv --json
python main.py --benchmark-dataset-solvers --size 9 --plot --csv --json
python main.py --test
```

Các lệnh trên minh họa cách sử dụng. Python từ 3.10 trở lên được yêu cầu theo tài liệu dự án. Gurobi và IBM CPLEX cần thư viện và quyền sử dụng phù hợp; lỗi license hoặc môi trường phải được phân biệt với kết quả không có nghiệm. Chức năng `--prepare-data` tải lại nguồn động, không bảo đảm tái tạo đúng snapshot 299 bản ghi đang đóng gói.

5.3. Kiểm thử và kiểm tra nghiệm

`test_encodings.py` duyệt 2^6 phép gán cho ALO, từng AMO và EO với năm phương pháp, đối chiếu tính khả thỏa với điều kiện số biến đúng. Đây là kiểm thử hữu hạn ở $n = 6$, chưa bao phủ mọi kích thước hoặc các trường hợp rỗng và đơn phần tử.

`test_solvers.py` kiểm tra bảng mẫu 9×9 với các solver; ở 4×4 kiểm tra SAT Product, CP-SAT và CBC; ở 16×16 kiểm tra SAT Product và CP-SAT. `test_dataset.py` kiểm tra chuyển đổi chuỗi–ma trận, định dạng dữ liệu và suy ra độ khó. Bộ kiểm tra nghiệm đối chiếu giá trị, hàng, cột, khối và gợi ý, không chứng minh nghiệm duy nhất. Các bài kiểm tra solver thương mại có cơ chế bỏ qua khi lỗi môi trường; do đó cần xem log cụ thể, không suy ra mọi solver đã vượt qua kiểm thử chỉ từ việc chương trình kết thúc. Báo cáo mô tả phạm vi mã kiểm thử hiện có, không công bố một lần chạy kiểm thử mới.

6. Kết quả thực nghiệm và Thảo luận (Experimental Evaluation)

6.1. Môi trường thực nghiệm & Bộ dữ liệu Benchmark

- **Phần cứng**: Intel Core i5-1235U (thế hệ 12), 16 GB RAM, hệ điều hành Windows x64.
- **Môi trường phần mềm**: Python 3.14, thư viện PySAT (python-sat 1.9), OR-Tools 9.15, Gurobi 13.0, CPLEX 22.2 và PuLP 3.3.
- **Dữ liệu Benchmark**: 299 bản ghi trong `data/benchmark_dataset.json`:
  - Bảng 9×9: 279 bản ghi, gồm *19 Easy, 77 Medium, 99 Hard, 84 Expert*; dùng cho Bảng 3 và Hình 1–3.
  - Bảng 4×4: 10 đề tự sinh.
  - Bảng 16×16: 10 đề tự sinh, tỷ lệ ô giữ lại được cấu hình từ 18% đến 55%. Bảng 2 trình bày cả nhóm 9×9 và 16×16.

6.2. Nguồn, chuẩn hóa và đặc điểm dữ liệu

Snapshot có 131 bản ghi Norvig (70 top95, 50 easy50, 11 hardest), 148 bản ghi Sudoku Exchange (43 easy, mỗi nhóm medium/hard/diabolical có 35), và 20 bản ghi tự sinh. KaggleHub và Hugging Face là các nguồn loader hỗ trợ tùy chọn, không có bản ghi trong snapshot thực nghiệm hiện tại. Các nguồn trực tuyến được tham khảo tại `https://norvig.com/sudoku.html` và `https://github.com/grantm/sudoku-exchange-puzzle-bank`.

Schema thống nhất gồm `id`, `source`, `dataset`, `size`, `difficulty`, `clues_count`, `puzzle_string`, `grid` và các trường nghiệm tùy chọn. Ô trống có giá trị 0; chuỗi dùng ký hiệu chữ cho các giá trị lớn hơn 9. Snapshot hiện chưa có đáp án chuẩn trong trường `solution_grid`; tính hợp lệ được kiểm tra bằng luật Sudoku thay vì đối chiếu đáp án lưu sẵn.

Loader ưu tiên nhãn nguồn khi nhận diện được; nếu không, suy ra theo mật độ $\rho = g/N^2$: Expert khi $\rho \leq 0.27$, Hard khi $0.27 < \rho \leq 0.33$, Medium khi $0.33 < \rho \leq 0.42$, Easy khi $\rho > 0.42$. Nhãn của snapshot được dùng nguyên trạng khi phân tích; một số nhãn khác với quy tắc ưu tiên nguồn của loader hiện tại nên không được coi là một chuẩn độ khó thống nhất giữa các nguồn. Số gợi ý chỉ là chỉ báo, không xác định đầy đủ độ khó đối với solver.

Hai bản ghi `norvig_easy50_0009` và `norvig_easy50_0041` trùng đề; do đó có 298 đề phân biệt (278 đề 9×9), dù các thống kê cũ vẫn tính trên 299 bản ghi. Các đề 16×16 được sinh bằng cách xóa ô từ một nghiệm cơ sở với bộ sinh ngẫu nhiên seed 42. Cách này bảo đảm tồn tại ít nhất một nghiệm nhưng chưa bảo đảm nghiệm duy nhất hoặc độ đa dạng tương đương dữ liệu thu thập độc lập.

6.3. Định nghĩa phép đo và phạm vi số liệu

Benchmark encoding đo riêng thời gian sinh CNF và thời gian gọi `solver.solve()`; tổng là tổng hai pha này, không gồm khởi tạo solver, nạp CNF, giải mã hoặc kiểm tra nghiệm. Benchmark solver đo toàn bộ lời gọi hàm giải, bao gồm dựng mô hình và các chi phí nằm trong wrapper, nhưng không gồm bước kiểm tra nghiệm bên ngoài. Vì vậy hai loại thời gian không thể so sánh trực tiếp.

Các bảng thời gian và hình trình bày số liệu thực nghiệm đã ghi nhận. Mã hỗ trợ xuất CSV/JSON và kiểm tra nghiệm, nhưng bộ kết quả đi kèm chưa có đủ dữ liệu thô để xác minh độc lập toàn bộ thời gian và tỷ lệ thành công. Vì vậy, các con số được sử dụng ở mức mô tả, chưa phải kết quả tái lập độc lập. Quy mô CNF có thể đối chiếu bằng công thức và đầu vào.

6.4. Đánh giá quy mô mô hình SAT Encodings (Variables vs Clauses)

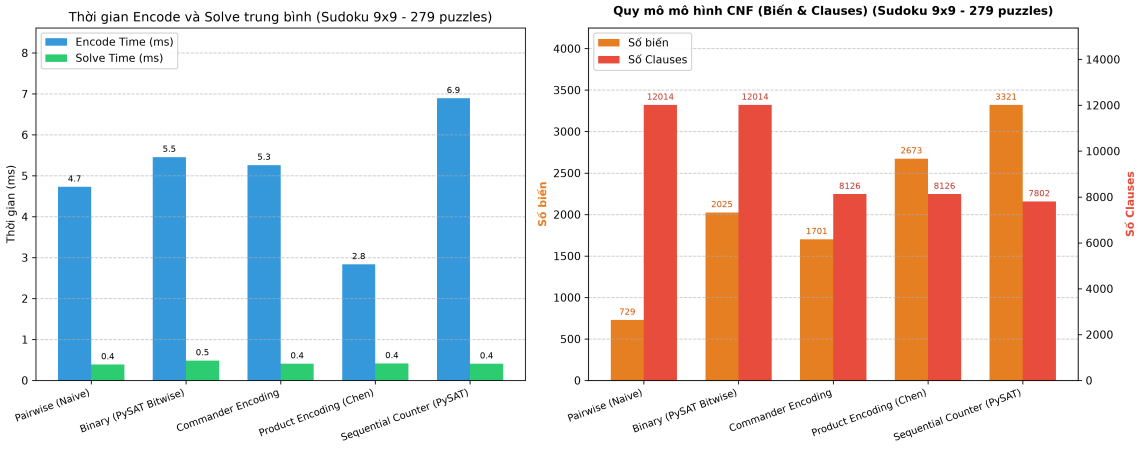
Bảng 2 trình bày hai nhóm kích thước; Hình 1 chỉ dùng nhóm 9×9. Với g gợi ý và $a(N)$ mệnh đề AMO mỗi nhóm, tổng CNF là $4N^2[1 + a(N)] + g$. Số mệnh đề trung bình được làm tròn đến số nguyên.

Phân tích:

- *Về số mệnh đề*: Pairwise sinh khoảng 12,014 mệnh đề ở 9×9 và 123,987 ở 16×16. Ở $N = 9$, Binary bằng Pairwise vì $\binom{9}{2} = 36 = 9\lceil \log_2 9 \rceil$; ở $N = 16$, Binary giảm khoảng 46%. Commander, Product và Sequential Counter giảm khoảng 32–35% ở 9×9 và 58–63% ở 16×16. Tính trên toàn bảng, Pairwise có quy mô $O(N^4)$, Binary là $O(N^3 \log N)$ và các AMO tuyến tính cho quy mô $O(N^3)$.
- *Về thời gian*: Trong số liệu nhóm 9×9, mã hóa mất khoảng 2.8–6.9 ms, còn gọi giải mất 0.4–0.5 ms. Product cho kết quả tốt ở phép đo hai pha. Chưa thể quy chênh lệch cho riêng unit propagation hoặc độ phức tạp tiệm cận vì cách cài đặt và các chi phí ngoài timer cũng ảnh hưởng đến thực thi.

Bảng 2
Quy mô trung bình số biến và mệnh đề sinh ra giữa 5 phương pháp AMO trên Sudoku 9×9 (279 câu đố) và 16×16 (10 câu đố).

Phương pháp AMO	Sudoku 9x9		Sudoku 16x16	
	Số biến	Số Clauses	Số biến	Số Clauses
Pairwise (Naive)	729	12,014	4,096	123,987
Binary (Bitwise)	2,025	12,014	8,192	66,643
Commander	1,701	8,126	11,264	52,307
2-Product	2,673	8,126	12,288	46,163
Sequential Counter	3,321	7,802	19,456	46,163



Hình 1: Thời gian mã hóa, gọi solve() và quy mô CNF trên 279 bản ghi 9×9; tổng hai pha không gồm khởi tạo và nạp CNF.

6.5. Phân tích độ nhạy cảm theo cấp độ khó (Difficulty Analysis)

Hình 2 biểu diễn tổng thời gian (mã hóa + giải) trung bình của 5 phương pháp SAT Encoding qua 4 cấp độ khó: *Easy* → *Medium* → *Hard* → *Expert*.

Nhận xét:

- **Khác biệt giữa các encoding**: Product có tổng thời gian mã hóa và gọi giải tương đối thấp ở cả bốn nhóm, khoảng 2.8–4.1 ms. Sequential Counter nằm ở mức cao hơn, khoảng 6.5–8.6 ms, dù sinh ít mệnh đề. Quan sát này gợi ý rằng số mệnh đề chỉ phản ánh một phần chi phí; cách sinh CNF và số biến phụ cũng là những yếu tố đáng xem xét.

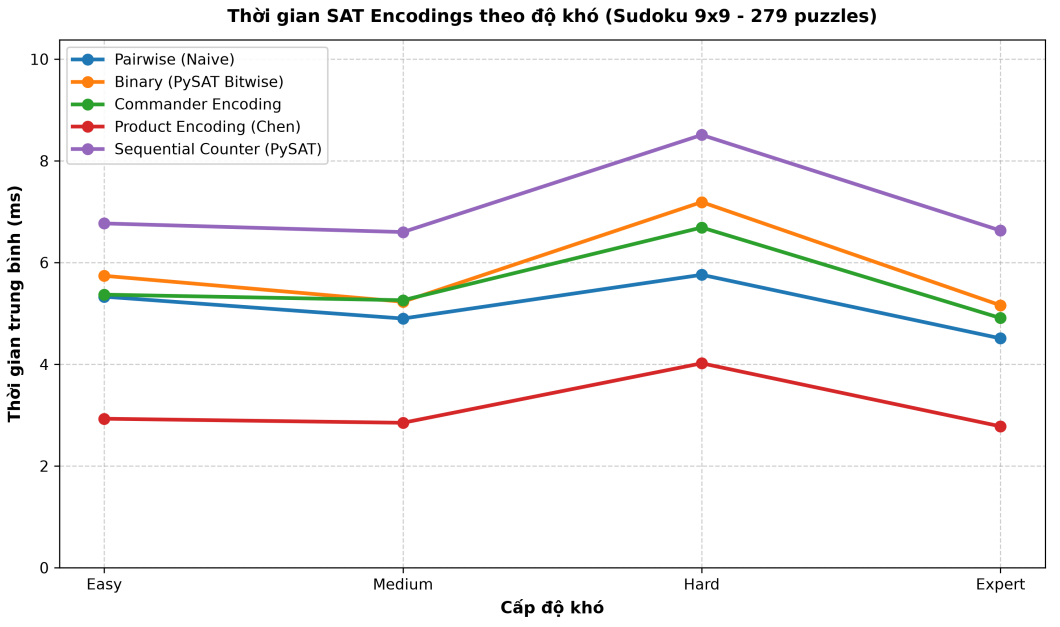
- **Xu hướng theo nhãn độ khó**: Thời gian trung bình không tăng đều từ Easy đến Expert; trong hình, nhóm Hard có giá trị cao hơn nhóm Expert. Vì nhãn được tổng hợp từ nhiều nguồn và một phần dựa trên mật độ gợi ý, thứ tự các nhãn không nhất thiết tương ứng với mức độ khó đối với Glucose3.

- **Vai trò của hai pha**: Trên nhóm bảng 9×9, thời gian mã hóa chiếm phần lớn tổng hai pha, trong khi thời gian gọi giải trung bình ở mức khoảng 0.4–0.5 ms. Vì vậy, khác biệt về quá trình tìm nghiệm có thể chưa thể hiện rõ trên đường tổng thời gian. Việc xem riêng pha giải sẽ giúp bổ sung góc nhìn về ảnh hưởng của cấu trúc từng đề.

- Nhìn chung, hình cho thấy các encoding có mức chi phí khác nhau nhưng chưa thể hiện một quan hệ đơn giản giữa nhãn độ khó và thời gian xử lý. Các ví dụ ở Mục 7 được dùng để quan sát thêm sự khác biệt trên từng đề, thay vì đại diện cho toàn bộ một cấp độ.

6.6. So sánh đối chuẩn hiệu năng giữa SAT Solver và các Solvers (CP, ILP/MIP)

Bảng 3 và Hình 3 minh họa kết quả đo lường thời gian thực thi trung bình giữa các công cụ giải khác nhau trên cùng tập dữ liệu chuẩn hóa.



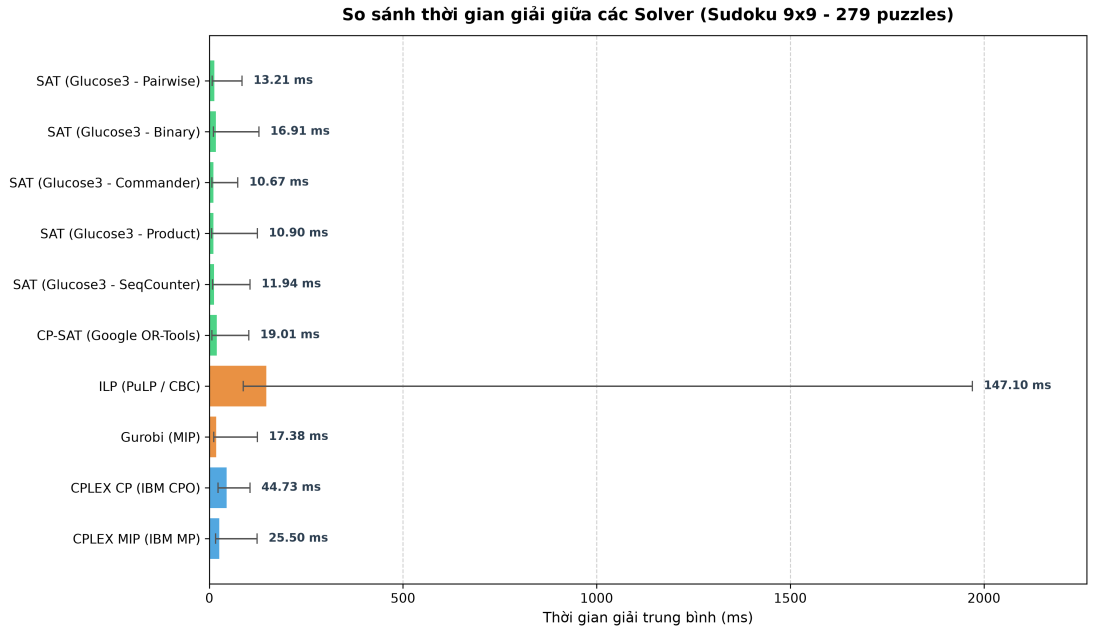
Hình 2: Xu hướng biến thiên tổng thời gian (mã hóa + giải) của 5 kỹ thuật SAT Encoding theo các cấp độ khó (Sudoku 9x9).

Bảng 3
Thời gian toàn bộ hàm giải trên 279 bản ghi 9×9; tỷ lệ nghiệm hợp lệ theo số liệu đã ghi nhận.

Bộ giải (Solver Engine)	Trường phái	Thời gian TB (ms)	Tỷ lệ thành công
SAT (Glucose3 - Commander)	SAT	**10.67**	100%
SAT (Glucose3 - Product)	SAT	**10.90**	100%
SAT (Glucose3 - SeqCounter)	SAT	**11.94**	100%
SAT (Glucose3 - Pairwise)	SAT	**13.21**	100%
SAT (Glucose3 - Binary)	SAT	**16.91**	100%
Gurobi Optimizer	MIP	17.38	100%
Google OR-Tools CP-SAT	CP / SAT Hybrid	19.01	100%
IBM CPLEX MIP	MIP	25.50	100%
IBM CPLEX CP Optimizer	CP	44.73	100%
ILP (PuLP / CBC)	ILP / Branch-and-Cut	147.10	100%

Thảo luận:

1. **Các cấu hình SAT**: Thời gian trung bình nằm trong khoảng 10.67–16.91 ms. Commander và Product có kết quả khá gần nhau (10.67 và 10.90 ms), nên có thể xem chúng là hai lựa chọn đáng chú ý trong phép đo này, thay vì nhấn mạnh thứ hạng từ một chênh lệch nhỏ.
2. **Đối chiếu với CP và MIP**: Gurobi (17.38 ms) và CP-SAT (19.01 ms) có thời gian cùng bậc độ lớn với nhóm SAT; Gurobi khá gần SAT Binary (16.91 ms). CPLEX MIP, CP Optimizer và CBC có thời gian cao hơn trong bảng, nhưng đều được ghi nhận tìm được nghiệm hợp lệ trên tập khảo sát. Kết quả cho thấy cả ba hướng mô hình hóa đều sử dụng được cho các đề 9×9 này, với chi phí thực thi khác nhau.
3. **Liên hệ với phép đo encoding**: Product có kết quả thuận lợi khi xét riêng mã hóa cộng gọi giải ở Mục 6.5, còn Commander có giá trị trung bình thấp hơn một chút khi đo toàn bộ hàm giải. Hai quan sát này không mâu thuẫn: phép đo toàn bộ còn tính khởi tạo solver, nạp CNF và giải mã. Việc chọn encoding vì thế nên gắn với công đoạn hoặc mục tiêu sử dụng cần quan tâm.
4. **Cách đọc chênh lệch thời gian**: CBC có khoảng cách rõ hơn so với các cấu hình còn lại, trong khi một số cặp solver chỉ chênh vài mili-giây. Các giá trị phản ánh đồng thời cách dựng mô hình, lớp giao tiếp Python và bộ



Hình 3: Thời gian thực thi các hàm giải trên 279 bản ghi Sudoku 9×9.

máy giải, nên phù hợp để so sánh các cài đặt trong dự án hơn là đánh giá riêng thuật toán bên trong từng solver. Đo lặp và tách thời gian dựng mô hình khỏi tìm kiếm sẽ giúp làm rõ hơn những khác biệt này.

7. Nghiên cứu trường hợp thực nghiệm (Experimental Case Studies)

Các bảng và hình tổng hợp dùng tập con ghi tại chú thích, không phải tất cả đều lấy trung bình trên 299 bản ghi. Mục này giữ lại các số liệu từng đề để minh họa kích thước, nhãn độ khó và encoding.

Phạm vi kiểm chứng. Các đề được định danh cụ thể trong từng bảng. Thời gian case study là số liệu tham khảo đã ghi nhận; bộ mã và dữ liệu thô đi kèm chưa đủ để tái tạo việc chọn mẫu theo trung vị gợi ý hoặc xác nhận giao thức lấy cực tiểu 60 lần trong ba phiên. Do đó, các bảng không được dùng như ước lượng thời gian điển hình hay bằng chứng về độ ổn định. Số biến và mệnh đề có thể đối chiếu với đầu vào và công thức. Lấy cực tiểu nhiều lần cũng không bảo đảm loại bỏ hoàn toàn nhiễu.

7.1. Trường hợp theo kích thước bảng (4×4, 9×9, 16×16)

Ba câu đố đại diện lần lượt là `matrix4x4_0002` (4×4, 4 gợi ý), `norvig_top95_0053` (9×9, 26 gợi ý) và `hexadoku16x16_0005` (16×16, 77 gợi ý). Bảng 4 tổng hợp quy mô CNF và thời gian tương ứng.

Nhận xét:

- Pairwise sinh 123, 981 mệnh đề trên đề 16×16, gấp khoảng 2.69 lần Product hoặc Sequential Counter (46, 157). AMO Pairwise tăng $O(n^2)$ trong một nhóm; vì Sudoku có $4N^2$ nhóm nên tổng CNF tăng $O(N^4)$, so với $O(N^3)$ khi dùng AMO tuyến tính.
- Product có tổng hai pha thấp nhất trên hai đề 9×9 và 16×16 trong bảng. Một đề mỗi kích thước chưa đủ đại diện hiệu năng toàn nhóm.
- Với 4×4, Product sinh 10 mệnh đề AMO mỗi nhóm so với 6 của Pairwise; cộng ALO thì có 11 và 7 mệnh đề EO. Tổng lần lượt là $64 \times 11 + 4 = 708$ và $64 \times 7 + 4 = 452$. Ưu thế tiệm cận không bảo đảm ưu thế ở nhóm nhỏ.
- Mã hóa lâu hơn gọi giải ở các hàng trong bảng, nhưng tỷ trọng này không gồm nạp CNF và không thay thế phép đo toàn bộ hàm giải.

Bảng 4
Quy mô CNF và thời gian tham khảo đã ghi nhận trên ba đề. Seq. Counter là Sequential Counter; tổng chỉ gồm mã hóa và gọi solve().

Bảng	Puzzle	Gợi ý	Mã hóa AMO	Biến	Mệnh đề	Enc. (ms)	Solve (ms)	Tổng (ms)
4×4	`matrix4x4_0002`	4	Pairwise	64	452	0.17	0.05	0.22
			Binary	192	580	0.58	0.07	0.65
			Commander	128	580	0.34	0.07	0.41
			2-Product	320	708	0.33	0.08	0.41
			Seq. Counter	256	580	0.73	0.06	0.79
9×9	`norvig_top95_0053`	26	Pairwise	729	12,014	2.80	0.45	3.25
			Binary	2,025	12,014	6.81	0.53	7.34
			Commander	1,701	8,126	3.05	0.43	3.48
			2-Product	2,673	8,126	2.50	0.43	**2.93**
			Seq. Counter	3,321	7,802	5.50	0.38	5.88
16×16	`hexadoku16x16_0005`	77	Pairwise	4,096	123,981	30.12	4.10	34.22
			Binary	8,192	66,637	30.14	2.78	32.92
			Commander	11,264	52,301	20.51	3.26	23.77
			2-Product	12,288	46,157	15.63	2.24	**17.87**
			Seq. Counter	19,456	46,157	30.73	3.53	34.26

7.2. Trường hợp theo cấp độ khó

Hai bảng 5 và 6 trình bày kết quả trên bốn câu đố 9×9 đại diện cho bốn cấp độ khó, với số gợi ý giảm dần từ 36 (Easy) xuống 23 (Expert).

Bảng 5
Tổng thời gian (mã hóa + giải, ms) trên bốn câu đố 9×9 đại diện: `norvig_easy50_0005` (Easy), `norvig_easy50_0003` (Medium), `norvig_top95_0012` (Hard) và `norvig_hardest_0002` (Expert).

Độ khó	Gợi ý	Pairwise	Binary	Commander	2-Product	Seq. Counter
Easy	36	3.23	7.85	3.81	3.27	5.00
Medium	28	3.21	6.20	3.52	3.50	6.44
Hard	25	3.54	7.01	4.03	3.63	7.01
Expert	23	4.13	8.32	5.53	4.90	7.37

Bảng 6
Thời gian giải thuần túy (ms, chỉ tính pha `Glucose3.solve()`) trên cùng bốn câu đố 9×9 đại diện theo cấp độ khó.

Độ khó	Gợi ý	Pairwise	Binary	Commander	2-Product	Seq. Counter
Easy	36	0.32	0.48	0.32	0.33	0.21
Medium	28	0.34	0.40	0.27	0.33	0.26
Hard	25	0.47	0.63	0.66	0.42	0.78
Expert	23	0.67	1.14	1.52	1.60	1.36

Nhận xét:

- Từ đề Easy đến Expert, Product tăng 3.27 $\rightarrow$ 4.90 ms, Sequential Counter 5.00 $\rightarrow$ 7.37 ms và Pairwise 3.23 $\rightarrow$ 4.13 ms. Dãy số không tăng đơn điệu ở mọi phương pháp; Product không thấp nhất trong cả bốn hàng.
- Tỷ số thời gian gọi giải Expert/Easy khoảng 2.1–6.5 lần, riêng Sequential Counter là $1.36/0.21 \approx 6.5$. Bốn đề khác nhau chưa đủ để tách ảnh hưởng số gợi ý khỏi cấu trúc đề và nguồn dữ liệu.
- Các bảng từng đề và Hình 2 dùng tập quan sát khác nhau nên xu hướng không nhất thiết trùng nhau. Chưa có dữ liệu thô để quy khác biệt cho GC, turbo boost hay một ngoại lệ cụ thể. Cần đo lặp có kiểm soát trước khi kết luận độ nhạy theo độ khó.

7.3. Trường hợp theo kỹ thuật mã hóa

Bảng 7 phân rã cấu trúc file CNF của câu đố 9×9 đại diện (26 gợi ý) theo từng kỹ thuật mã hóa.

Bảng 7
Phân rã cấu trúc CNF của câu đố `norvig_top95_0053` (9×9, 26 gợi ý): 324 ràng buộc Exactly-One ($4N^2$) tương ứng 324 mệnh đề ALO, 26 mệnh đề đơn nguyên từ gợi ý, phần còn lại là AMO.

Phương pháp	Biến	Biến phụ	Mệnh đề	ALO	AMO	Gợi ý	Encode (ms)	Solve (ms)
Pairwise (Naive)	729	0	12,014	324	11,664	26	2.80	0.45
Binary (Bitwise)	2,025	1,296	12,014	324	11,664	26	6.81	0.53
Commander	1,701	972	8,126	324	7,776	26	3.05	0.43
2-Product	2,673	1,944	8,126	324	7,776	26	2.50	0.43
Sequential Counter	3,321	2,592	7,802	324	7,452	26	5.50	0.38

Nhận xét:

- Có 324 EO, 324 ALO và 26 mệnh đề gợi ý. AMO mỗi nhóm sinh lần lượt 36, 36, 24, 24, 23 mệnh đề, phù hợp các công thức đã trình bày. Đây là kiểm tra quy mô, chưa phải chứng minh đúng đắn ngữ nghĩa.
- Biến phụ lần lượt là $324 \times 4 = 1,296$ (Binary), $324 \times 3 = 972$ (Commander ở kích thước này), $324 \times 6 = 1,944$ (Product), $324 \times 8 = 2,592$ (Sequential Counter). Với Commander lớn hơn cần tính cả các tầng đệ quy.
- Product mã hóa nhanh nhất (2.50 ms), nhưng Sequential Counter gọi giải nhanh nhất (0.38 ms). Khác biệt giữa gọi PySAT và tự sinh Python chưa đủ để kết luận chi phí thư viện là nguyên nhân chính.

Bảng 8 minh họa đề có 26 gợi ý và một nghiệm hợp lệ được ghi nhận với Glucose3, mã hóa Product. Nghiệm thỏa hàng, cột, khối và gợi ý; chưa khẳng định duy nhất vì quy trình hiện chỉ tìm một nghiệm, không loại nghiệm đó để kiểm tra nghiệm thứ hai.

Bảng 8
Câu đố 9×9 đại diện `norvig_top95_0053` (26 gợi ý) và nghiệm do Glucose3 với mã hóa 2-Product tìm được.

Đề bài

	2					5	9	3
8			5			4	6	
9	4			6				8
		2		3				
	6			8		7	3	
7			2					
				4		3	8	
	7					6		
								5

Nghiệm

1	2	6	4	7	8	5	9	3
8	3	7	5	9	2	4	6	1
9	4	5	3	6	1	2	7	8
4	1	2	9	3	7	8	5	6
5	6	9	1	8	4	7	3	2
7	8	3	2	5	6	9	1	4
2	5	1	6	4	9	3	8	7
3	7	4	8	1	5	6	2	9
6	9	8	7	2	3	1	4	5

8. Kết luận và Hướng phát triển (Conclusion)

Bài tập lớn xây dựng hệ thống Sudoku bằng SAT và tích hợp CP/MIP để so sánh. Pairwise, Commander và Product được tự cài đặt; Binary và Sequential Counter sử dụng PySAT. Công thức CNF thể hiện sự đánh đổi giữa biến phụ và mệnh đề khi tăng kích thước.

Trong số liệu đã ghi nhận, Product có chi phí hai pha thấp ở nhiều cấu hình, còn Commander có thời gian toàn bộ hàm giải trung bình thấp nhất ở bảng 9×9. Không có encoding tốt nhất độc lập với tiêu chí đo, kích thước và cài đặt. Kết quả chưa chứng minh ưu thế tổng quát của SAT trước CP/MIP.

Hướng phát triển gần nhất là hoàn thiện kiểm tra đầu vào, khử trùng dữ liệu, kiểm tra duy nhất, bổ sung đo lặp và dữ liệu thô. Sau đó có thể khảo sát Diagonal hoặc Killer Sudoku bằng ràng buộc phù hợp; SMT và MaxSAT là hướng bổ sung, không bắt buộc cho mọi biến thể.

Tính sẵn có của mã nguồn và dữ liệu (Data Availability)

Mã nguồn, snapshot 299 bản ghi, script benchmark và biểu đồ được lưu tại dự án dưới đây.

https://github.com/nmson8206/Sudoku_prj

Tài liệu

Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers. IJCAI 9, 399–404.

Chen, J., 2010. A new sat encoding of the at-most-one constraint, in: Constraint Programming and Decision Making: Workshop at the European Conference on Artificial Intelligence (ECAI).

Frisch, A.M., Peugniez, N., Doggett, A.J., Nightingale, P., 2008. Effective propositional encodings of constraint satisfaction problems. Artificial Intelligence 172, 1961–1994.

Gurobi Optimization, LLC, 2024. Gurobi optimizer reference manual. Online: https://www.gurobi.com .

IBM Corporation, 2024. Ibm ilog cplex optimization studio cplex user's manual. Online: https://www.ibm.com/analytics/cplex-optimizer .

Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. Pysat: A python toolkit for prototyping with sat oracles, in: International Conference on Theory and Applications of Satisfiability Testing, Springer. pp. 428–437.

Klieber, W., Kwon, G., 2007. Efficient cnf encoding for selecting 1 from n objects, in: International Conference on Theory and Applications of Satisfiability Testing, Springer. pp. 208–214.

Norvig, P., 2006. Solving every sudoku puzzle. Online essay: http://norvig.com/sudoku.html .

Perron, L., Furnon, V., 2024. Google or-tools: Cp-sat solver reference. Google Developers .

Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints. Principles and Practice of Constraint Programming-CP 2005 , 827–831.